

Databases 2 - exam - June 4, 2025 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency control (10 points)

Given the resource hierarchy $X(Y(A,B), W(C,D))$, describe the behavior of the following arrival sequence of requests managed by a scheduler that applies hierarchical locking. Suppose that locks are released after the commit of each transaction, which occurs immediately after its final operation in the schedule. A transaction that is waiting for a lock does not proceed with its subsequent operations.

r1(C) r2(D) w1(A) w2(W) r2(A) r3(X) w1(Y)

Present your answer in a step-by-step table where each row corresponds to an operation or a commit, and each column indicates the lock state of each resource in the hierarchy.

Solution.

Op.	X	Y	A	B	W	C	D
r1(C)	ISL ₁	-	-	-	ISL ₁	SL ₁	-
r2(D)	ISL _{1,2}	-	-	-	ISL _{1,2}	SL ₁	SL ₂
w1(A)	ISL ₂ , IXL ₁ (upgr ₁)	IXL ₁	XL ₁	-	ISL _{1,2}	SL ₁	SL ₂
w2(W)	IXL _{1,2} (upgr ₂)	IXL ₁	XL ₁	-	ISL _{1,2} (Conflict - T2 waits $\leftarrow XL_2$)	SL ₁	SL ₂
r2(A)	T2 is waiting						
r3(X)	IXL _{1,2} (Conflict - T3 waits! $\leftarrow SL_3$)	IXL ₁	XL ₁	-	ISL _{1,2} $\leftarrow XL_2$	SL ₁	SL ₂
w1(Y)	IXL _{1,2} $\leftarrow SL_3$	XL ₁ (upgr)	XL ₁	-	ISL _{1,2} $\leftarrow XL_2$	SL ₁	SL ₂
T1 commits	IXL ₂ $\leftarrow SL_3$	-	-	-	ISL ₂ $\leftarrow XL_2$	-	SL ₂
w2(W)	IXL ₂ $\leftarrow SL_3$	-	-	-	XL ₂ (upgr)	-	SL ₂
r2(A)	IXL ₂ $\leftarrow SL_3$	ISL ₂	SL ₂	-	XL ₂	-	SL ₂
T2 commits	-	-	-	-	-	-	-
r3(X)	SL ₃	-	-	-	-	-	-
T3 commits	-	-	-	-	-	-	-

B. Trigger (10 points)

Given the following relational schema:

EMP(empID, name, deptID)

PROJECT(projID, funding, deptID)

DEPT(deptID, name, address)

PROJECTS-INCOME(deptID, totAmt, perCapitaAmt)

An employee belongs to a single department and a project is associated with a single department. The database schema does not record the association of employees to projects.

Assume that:

- The primary keys are not updated.
- The funding and department (deptID) of a project cannot be NULL or modified.
- Projects and departments cannot be deleted.

Write a set of triggers that incrementally maintain the values of:

- totAmt (= sum of all the funds of all the projects of a department)
- perCapitaAmt (= totAmt/numEmp, where numEmp is the number of employees in the department)

List all the events that the triggers should process to maintain the content of the PROJECTS-INCOME table and define the code of such triggers.

Solution.

1. Insertion of an employee:

```
CREATE TRIGGER EmpInsert
AFTER INSERT ON EMP
FOR EACH ROW
BEGIN
    UPDATE PROJECTS-INCOME
    SET perCapitaAmt = totAmt / (SELECT COUNT(*) FROM EMP WHERE deptID = NEW.
        deptID)
    WHERE deptID = NEW.deptID;
END
```

2. Deletion of an employee:

```
CREATE TRIGGER EmpDelete
AFTER DELETE ON EMP
FOR EACH ROW
BEGIN
    UPDATE PROJECTS-INCOME
    SET perCapitaAmt = totAmt / (SELECT COUNT(*) FROM EMP WHERE deptID = OLD.
        deptID)
    WHERE deptID = OLD.deptID;
END
```

Alternative solution with check on the number of employees:

```
CREATE TRIGGER EmpDelete
AFTER DELETE ON EMP
FOR EACH ROW
BEGIN
    DECLARE emp_count INT;

    SELECT COUNT(*) INTO emp_count FROM EMP WHERE deptID = OLD.deptID;

    IF emp_count = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot delete the last
            employee in the department.';
    END IF;
```

```

UPDATE PROJECTS-INCOME
SET perCapitaAmt = totAmt / emp_count
WHERE deptID = OLD.deptID;
END

```

3. Update of deptID on EMP:

```

CREATE TRIGGER DeptUpdateForEmp
AFTER UPDATE ON EMP
FOR EACH ROW
BEGIN
    IF OLD.deptID != NEW.deptID THEN
        -- First update the old department
        UPDATE PROJECTS-INCOME
        SET perCapitaAmt = totAmt / (SELECT COUNT(*) FROM EMP WHERE deptID = OLD.
            deptID)
        WHERE deptID = OLD.deptID;

        -- Then update the new department
        UPDATE PROJECTS-INCOME
        SET perCapitaAmt = totAmt / (SELECT COUNT(*) FROM EMP WHERE deptID = NEW.
            deptID)
        WHERE deptID = NEW.deptID;
    END IF;
END

```

4. Insertion of a project

```

CREATE TRIGGER ProjectInsert
AFTER INSERT ON PROJECT
FOR EACH ROW
BEGIN
    -- Update the total amount
    UPDATE PROJECTS-INCOME
    SET totAmt = totAmt + NEW.funding
    WHERE deptID = NEW.deptID;

    -- Update the perCapita amount
    UPDATE PROJECTS-INCOME
    SET perCapitaAmt = totAmt / (SELECT COUNT(*) FROM EMP WHERE deptID = NEW.
        deptID)
    WHERE deptID = NEW.deptID;
END

```

An alternative solution with two triggers, one reacting to the insertion of a new project and one reacting to the update of the total amount from projects of a department by the first trigger is also acceptable.

C. Physical DB (12 points - 4 points for each scenario)

A ticketing company manages the sale of tickets for special (non-tour) concerts featuring various pop singers. The company stores a table `Ticket` (`TicketId*`, `PurchaseDate`, `ConcertName`, `SeatSection`, `Customer`, `Price`) containing 500K tickets sold over 8 years. The table is stored in a sequential structure occupying 20K blocks. Table `Concert` (`ConcertName*`, `Singer`, `Genre`, `Organizer`, `Description`) contains 800 rows, distributed across 30 leaf nodes in a small B+ tree built on `ConcertName`, having only 2 levels. $\text{Val}(\text{PurchaseDate}) = 2,000$ and $\text{val}(\text{Singer}) = 100$.

Evaluate the cost in terms of I/O accesses of the following query, in the three following scenarios:

```
// Retrieve the concert by Taylor Swift with the lowest number of tickets purchased
// during the year 2024
SELECT ConcertName, COUNT(*) AS TicketsSold
FROM Ticket JOIN Concert ON Ticket.ConcertName = Concert.ConcertName
WHERE Singer = 'Taylor Swift' AND PurchaseDate BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY ConcertName
ORDER BY TicketsSold ASC
LIMIT 1;
```

Scenarios:

1. The query runs without any additional indexes on the tables. For this first scenario, consider strategies that start from a scan of the concerts.
2. A hash index is created on the `Singer` column of the `Concert` table. This occupies 5 blocks and has no overflow chains.
3. On scenario 1, a B+ tree index is created on the composite key (`PurchaseDate`, `ConcertName`) in the `Ticket` table. The B+ tree index has 3 levels and 4K leaf nodes.

For each scenario,

- Briefly describe the strategy used to evaluate the query. For example, specify the join method, how data is accessed (e.g., scan&look up), etc. and when conditions are evaluated.
- Indicate the number of blocks or index leaf nodes that must be accessed.
- Provide all computations and assumptions (e.g., data distribution, number of tuples per block, etc.)
- Explain how the group by and the order by are performed.
- Write the complete I/O cost formulas, without caching and with caching (if applicable). Provide the final numerical cost estimate.

Solution.

1. Scenario No auxiliary

Scan Concert and lookup Ticket. Scan `Concert` and filter the tuples where the singer is “Taylor Swift”. For each of her concerts, lookup the associated tickets in the `Ticket` table and evaluate the condition on the date.

$$\begin{aligned} \text{Cost} &= 1 \text{ (B+ root)} + 30 \text{ (full scan of B+ leaf nodes)} + \\ &\quad 8 \text{ (concerts by Taylor Swift computed as } 800 \text{ concert tuples}/100 \text{ singers)} \\ &\quad * 20K \text{ (scan Ticket to compute the join and evaluate the condition on the date)} \\ &\simeq \mathbf{160K} \end{aligned}$$

To compute the count(*) for each of the 8 concerts, a counter is kept and tickets for that concert are counted during the ticket scan. The resulting 8 values can be sorted in main memory, and the concert with the lowest count can be returned. These operations do not require additional I/O accesses.

With caching The 8 blocks of the concerts of Taylor Swift can be cached:

Cost = 1 + 30 + 20K = **20K I/O** accesses

2. Scenario with hash index on Singer

The hash index on the singer attribute is exploited: “Taylor Swift” is searched in the hash index (cost 1 I/O); it is associated with 8 concerts, therefore 8 pointers will be followed to the 8 blocks containing the data of the concerts, specifically to obtain the ConcertName. From this point onward, the query proceeds as in the previous plan: for each concert, the Ticket table is scanned to find tickets corresponding to 2024 purchases:

Cost = 1 (hash index) + 8 (retrieve full concert tuples) + 8 * 20K (full scan of Ticket for each concert) = **160K I/O //without caching**

Cost = 1 (hash index) + 8 (retrieve full concert tuples) + 20K (single full scan of Ticket) = **20K I/O //with caching**

This hash index slightly improves the previous plan. *If tables are small (the Concert table occupies 30 blocks), indexes should be avoided.*

3. Scenario with B+ index on (PurchaseDate, ConcertName)

Like in Scenario 1, we begin by scanning the Concert table and filtering the tuples where the singer is “Taylor Swift”. This results in 8 concert names. Next, we use the B+ tree index on (PurchaseDate, ConcertName) to retrieve tickets purchased in dates between 01-01-2024 and 31-12-2024. Assuming purchases are uniformly distributed over 8 years, the 2024 subset spans approximately 4K / 8 = 500 leaf nodes in the B+ tree. Within these 500 leaf nodes, we identify tickets associated with the 8 Taylor Swift concerts. Notice that both the join condition (on ConcertName) and the date filter can be evaluated directly within the B+ tree without accessing the full ticket tuples stored in the sequential structure.

The number of blocks to visit in the B+(PurchaseDate, ConcertName) are:

2 intermediate index nodes
+ 4K leaf nodes/8 years
= 2 + 500 leaf nodes for 2024

Cost = 1 + 30 (scan concert) + 8 * (2+500) = **4047 I/O // without caching**

If the 8 concerts are cached in memory and a counter is maintained in main memory for each concert, then the B+ blocks need to be read only once:

Cost = 1 + 30 (scan concert) + (2+500) = **533 I/O // with caching**

With a finer analysis, we can consider that in 2024 we have a total of 500K/8=62.5K tickets and 800/8=100 concerts, so the average number of tickets per concert is 62.5K/100=625 tickets/concert. They fit in 4K/500K*625=5(+1)=6 blocks. Considering that we do not need to access the primary structure, the cost is as follows:

Cost = 1 + 30 + 8 (2+6) = **95 I/O // without caching**